

Projeto Integrador: Plataforma de Integração de Dados IoT

Integração de Dados II — 2025/1

IoT Lakehouse Batch + Streaming Airflow Kafka Spark/Flink

1. Visão Geral

1.1 Objetivo

Desenvolver uma **plataforma completa de integração de dados** para um cenário de **IoT (Internet of Things)**, aplicando os conceitos estudados na disciplina: ingestão de dados de múltiplas fontes, armazenamento em lakehouse, processamento batch e streaming, orquestração de pipelines e consumo analítico.

1.2 Contexto do Projeto

Você foi contratado(a) como **Engenheiro(a) de Dados** de uma empresa de monitoramento industrial. A empresa possui sensores distribuídos em fábricas que coletam dados de temperatura, umidade, pressão e vibração. Seu desafio é construir a infraestrutura de dados que permita:

- **Ingerir** dados de sensores em tempo real e de sistemas legados
- **Armazenar** dados brutos e processados em um data lakehouse
- **Processar** dados em batch e streaming
- **Orquestrar** pipelines de forma confiável
- **Disponibilizar** dados para dashboards e alertas

1.3 Competências Desenvolvidas

Competência	Descrição
Modelagem de dados	Projetar schemas para dados semi-estruturados (NoSQL) e estruturados
Arquitetura de dados	Implementar arquitetura lakehouse com camadas medallion
Integração	Conectar múltiplas fontes de dados heterogêneas
Processamento	Desenvolver pipelines batch e streaming
Orquestração	Automatizar e monitorar workflows de dados
Governança	Aplicar boas práticas de qualidade e documentação

2. Formato e Organização

2.1 Composição das Equipes

- **Flexível**: 1 a 4 integrantes por equipe
- Equipes devem ser formadas até a **2ª semana de aula**

- Registro via formulário no Canvas

2.2 Estrutura de Entregas

Entrega	Peso	Prazo	Sugerido	Foco Principal
E1 — Fundação	10 pts	Semana 5		Infraestrutura, ingestão, modelagem NoSQL
E2 — Pipeline	10 pts	Semana 10		Lakehouse, processamento, orquestração
E3 — Integração Final	20 pts	Semana 15		Streaming, consumo, documentação, demo
Total	40 pts	—	—	

2.3 Repositório do Projeto

Cada equipe deve manter um repositório Git (GitHub/GitLab) com:

```
projeto-iot-dados/  
├── README.md                # Documentação principal  
├── docker-compose.yml      # Infraestrutura completa  
├── docs/  
│   ├── arquitetura.md      # Decisões arquiteturais (ADRs)  
│   ├── modelo-dados.md     # Modelagem de dados  
│   └── runbook.md          # Guia de operação  
├── src/  
│   ├── ingestao/           # Scripts de ingestão  
│   ├── processamento/     # Jobs Spark/Flink  
│   ├── dags/               # DAGs do Airflow  
│   └── streaming/          # Consumers Kafka  
├── data/  
│   ├── sample/             # Dados de exemplo  
│   └── schemas/            # Schemas Avro/JSON  
├── tests/                  # Testes automatizados  
└── notebooks/             # Análises exploratórias
```

3. Cenário de Negócio

3.1 Fontes de Dados

Fonte	Tipo	Formato	Volume	Frequência
Sensores IoT	Streaming	JSON	~1000 eventos/min	Tempo real
Sistema Legado (ERP)	Batch	CSV/PostgreSQL	~100K registros	Diário
Cadastro de Equipamentos	Referência	MongoDB	~10K documentos	Sob demanda
API de Manutenção	REST	JSON	~500 registros/dia	Horário

3.2 Dados dos Sensores (Streaming)

```
{  
  "sensor_id": "SENS-001-TEMP",  
  "equipment_id": "EQ-1234",  
  "factory_id": "FAB-SP-01",  
  "measurement_type": "temperature",  
  "value": 72.5,  
  "unit": "celsius",  
  "timestamp": "2025-03-15T14:30:00.123Z",  
  "quality": "good",  
  "metadata": {  
    "firmware_version": "2.1.0",  
    "battery_level": 85  
  }  
}
```

```
}  
}
```

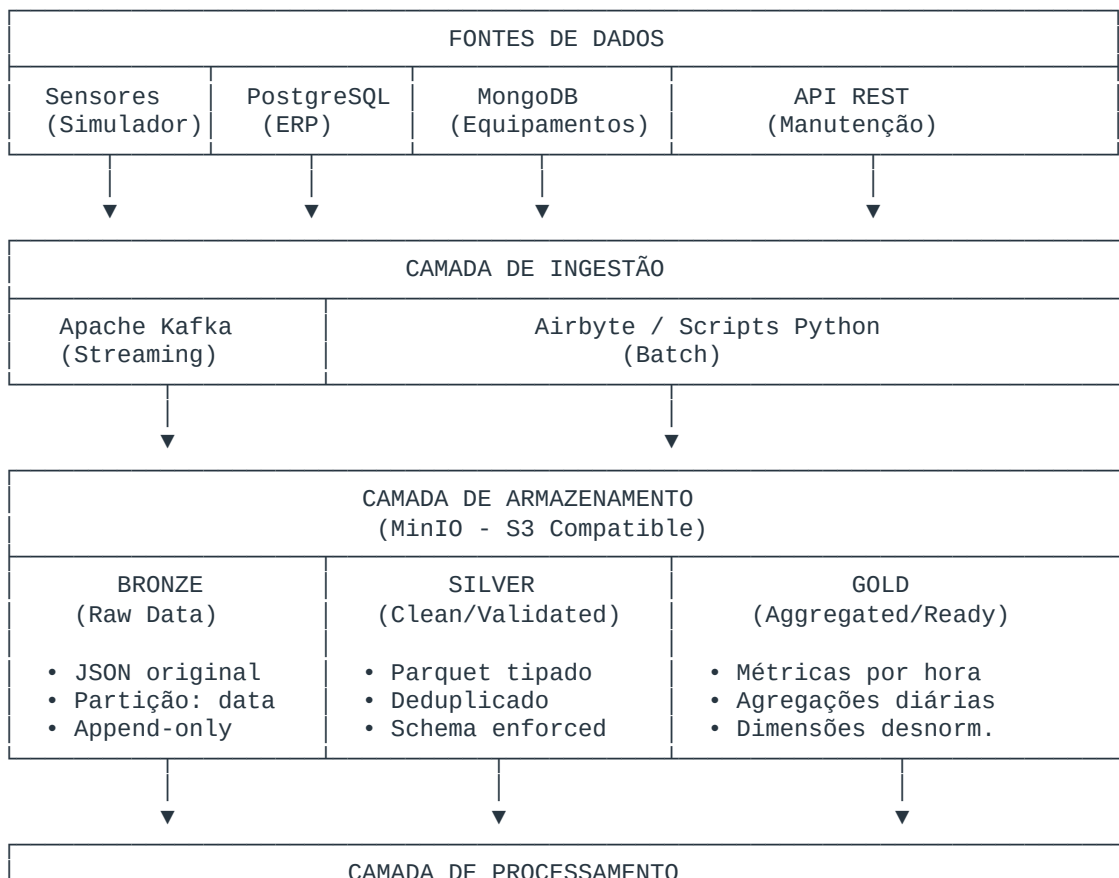
3.3 Cadastro de Equipamentos (MongoDB)

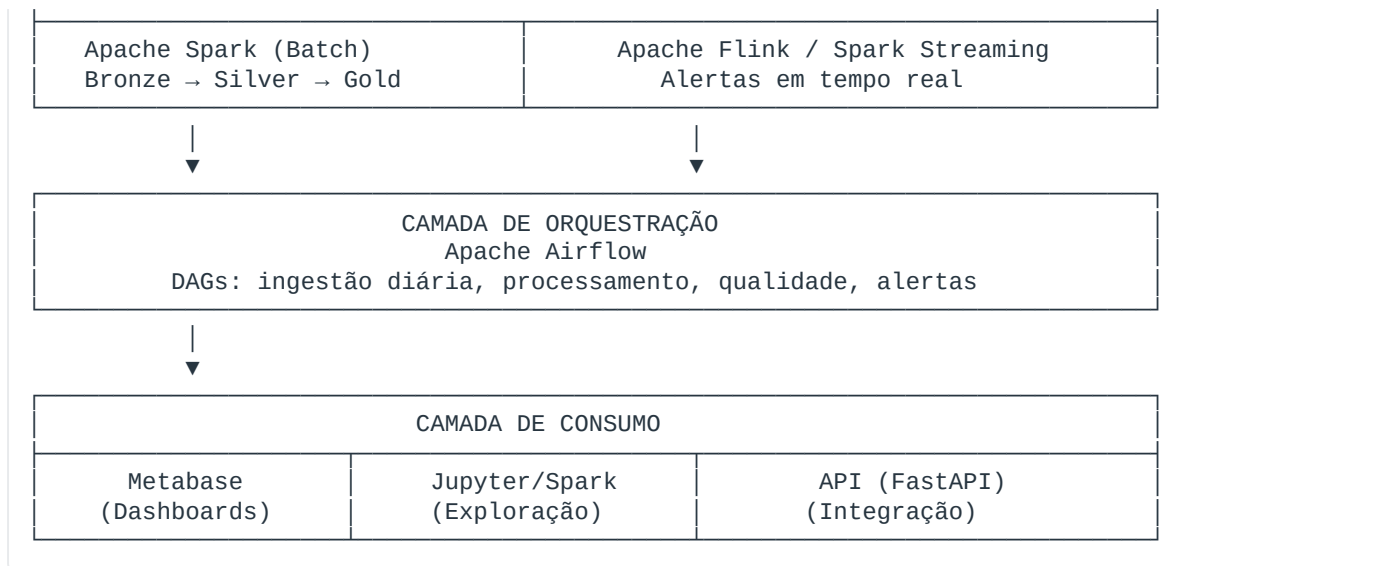
```
{  
  "_id": "EQ-1234",  
  "name": "Compressor Industrial A1",  
  "type": "compressor",  
  "factory": {  
    "id": "FAB-SP-01",  
    "name": "Fábrica São Paulo",  
    "location": {"lat": -23.5505, "lng": -46.6333}  
  },  
  "sensors": [  
    {"id": "SENS-001-TEMP", "type": "temperature", "range": {"min": 0, "max": 150}},  
    {"id": "SENS-002-VIBR", "type": "vibration", "range": {"min": 0, "max": 100}}  
  ],  
  "maintenance_schedule": "monthly",  
  "installed_at": "2023-06-15",  
  "status": "active"  
}
```

3.4 Requisitos de Negócio

1. **Dashboard Operacional:** visualizar métricas em near real-time (< 1 min de atraso)
2. **Alertas de Anomalia:** detectar leituras fora do range esperado
3. **Relatórios Diários:** agregações por fábrica, equipamento, tipo de sensor
4. **Histórico Completo:** manter dados brutos por 2 anos para auditoria
5. **Análise Preditiva:** dados preparados para modelos de ML (manutenção preditiva)

4. Arquitetura de Referência





5. Stack Tecnológica Obrigatória

Categoria	Tecnologia	Versão Mínima	Obrigatório
Containerização	Docker + Docker Compose	24.x / 2.x	✓ Sim
Streaming	Apache Kafka	3.x	✓ Sim
Orquestração	Apache Airflow	2.8+	✓ Sim
Processamento	Spark ou Flink	3.5+ / 1.18+	✓ Sim (escolher 1)
Object Storage	MinIO	latest	✓ Sim
NoSQL	MongoDB	6.x+	✓ Sim
SQL	PostgreSQL	15+	✓ Sim
Visualização	Metabase, Superset ou similar	—	Recomendado

5.1 Opcionais (Pontos Extras)

Tecnologia	Pontuação Extra	Descrição
Apache Iceberg/Delta Lake	+2 pts	Table format no lakehouse
dbt	+1 pt	Transformações documentadas
Great Expectations	+1 pt	Testes de qualidade de dados
Terraform/Pulumi	+1 pt	Infraestrutura como código
CI/CD (GitHub Actions)	+1 pt	Pipeline de deploy automatizado

6. Entrega 1 — Fundação (10 pontos)

6.1 Objetivo

Estabelecer a **infraestrutura base** do projeto, incluindo ambiente containerizado, fontes de dados configuradas, modelo NoSQL definido e ingestão inicial funcionando.

6.2 Requisitos Obrigatórios

#	Requisito	Peso
1.1	Docker Compose funcional com todos os serviços básicos (Kafka, MongoDB, PostgreSQL, MinIO)	2 pts
1.2	Simulador de sensores gerando dados para Kafka (mínimo 100 eventos/min)	2 pts
1.3	Modelagem NoSQL (MongoDB) para cadastro de equipamentos com documento de justificativa	2 pts
1.4	Consumer Kafka gravando dados brutos no MinIO (camada Bronze)	2 pts
1.5	Documentação : README com instruções de setup + ADR de modelagem	2 pts

6.3 Entregáveis

- Repositório Git com código fonte
- `docker-compose.yml` funcional
- Documento `docs/modelo-dados.md` com modelagem NoSQL justificada
- Demonstração: `docker-compose up` + dados fluindo para Bronze
- README com instruções de instalação e execução

6.4 Critérios de Avaliação E1

Critério	Insuficiente (0-40%)	Básico (41-70%)	Bom (71-90%)	Excelente (91-100%)
Infraestrutura	Não sobe ou erros críticos	Sobe com intervenção manual	Sobe com <code>docker-compose up</code>	Sobe + health checks + logs organizados
Simulador	Não gera dados	Gera dados fixos	Gera dados variados	Dados realistas + anomalias ocasionais
Modelagem NoSQL	Sem documento	Modelo básico sem justificativa	Modelo com embedding/referência justificado	Modelo otimizado + índices + evolução planejada
Ingestão Bronze	Não funciona	Grava JSON puro	Grava particionado por data	Particionado + schema registry + idempotente
Documentação	Inexistente	README básico	README + ADR	README detalhado + ADR + diagrama

7. Entrega 2 — Pipeline (10 pontos)

7.1 Objetivo

Implementar o **pipeline de processamento completo** com camadas Silver e Gold, orquestração via Airflow e primeiras visualizações.

7.2 Requisitos Obrigatórios

#	Requisito	Peso
2.1	Job Spark/Flink transformando Bronze → Silver (limpeza, tipagem, deduplicação)	2.5 pts
2.2	Job Spark/Flink agregando Silver → Gold (métricas horárias/diárias)	2 pts
2.3	DAGs Airflow orquestrando os jobs com schedule diário	2 pts
2.4	Ingestão batch do PostgreSQL (dados do ERP) via Airflow	1.5 pts
2.5	Dashboard básico (Metabase/Superset) com 3+ visualizações	2 pts

7.3 Entregáveis

- Jobs Spark/Flink em `src/processamento/`
- DAGs Airflow em `src/dags/`
- Script de ingestão do PostgreSQL
- Documento `docs/arquitetura.md` atualizado com fluxo de dados
- Screenshots ou export do dashboard
- Demonstração: execução de DAG + visualização de dados no dashboard

7.4 Critérios de Avaliação E2

Critério	Insuficiente (0-40%)	Básico (41-70%)	Bom (71-90%)	Excelente (91-100%)
Bronze → Silver	Não processa	Copia sem transformar	Limpa + tipagem	Dedup + validação + particionamento otimizado
Silver → Gold	Não agrega	Agregação simples	Múltiplas agregações	Agregações + dimensões + incremental
Airflow	Sem DAGs	DAG manual	DAG escalada	DAGs com retry, alertas, sensores
Ingestão Batch	Não funciona	Full load	Full load escalado	Incremental com CDC ou watermark
Dashboard	Inexistente	1-2 gráficos	3+ gráficos relevantes	Dashboard completo + filtros + drill-down

8. Entrega 3 — Integração Final (20 pontos)

8.1 Objetivo

Completar a plataforma com **processamento streaming**, **alertas**, **documentação completa** e **demonstração funcional** do sistema integrado.

8.2 Requisitos Obrigatórios

#	Requisito	Peso
3.1	Pipeline streaming processando eventos Kafka em near real-time (janelas de 1-5 min)	4 pts
3.2	Deteção de anomalias com alertas (valores fora do range do equipamento)	3 pts
3.3	Integração com API de manutenção (ingestão via Airflow)	2 pts

#	Requisito	Peso
3.4	Catálogo de dados documentado (schemas, linhagem, owners)	2 pts
3.5	Testes de qualidade de dados (mínimo 5 assertions)	2 pts
3.6	Documentação completa (arquitetura, runbook, ADRs)	3 pts
3.7	Apresentação/Demo ao vivo (15-20 min)	4 pts

8.3 Entregáveis

- Pipeline streaming em `src/streaming/`
- Sistema de alertas funcional
- Documento `docs/catalogo-dados.md`
- Testes em `tests/` com relatório de execução
- Documentação completa em `docs/`
- Apresentação (slides ou demo estruturada)
- Vídeo de backup (3-5 min) caso demo falhe

8.4 Roteiro da Apresentação Final

1. **Contexto** (2 min): problema de negócio, requisitos
2. **Arquitetura** (3 min): decisões técnicas, trade-offs
3. **Demo ao vivo** (10 min):
 - Subir ambiente (`docker-compose up`)
 - Mostrar dados fluindo (Kafka → Bronze → Silver → Gold)
 - Executar DAG no Airflow
 - Mostrar dashboard com dados
 - Provocar anomalia e mostrar alerta
4. **Desafios e Aprendizados** (3 min)
5. **Perguntas** (2 min)

8.5 Critérios de Avaliação E3

Critério	Insuficiente (0-40%)	Básico (41-70%)	Bom (71-90%)	Excelente (91-100%)
Streaming	Não funciona	Processa sem janelas	Janelas funcionais	Janelas + watermarks + late data handling
Alertas	Inexistente	Detecção básica	Detecção + notificação	Detecção inteligente + histórico + ações
Qualidade	Sem testes	1-2 testes	5+ testes	Testes integrados ao pipeline + relatórios
Documentação	Mínima	README + 1 doc	Docs completos	Docs + diagramas + runbook operacional
Demo	Não funciona	Funciona parcialmente	Fluxo completo	Fluxo completo + resiliência + improviso

9. Rubrica Geral de Avaliação

9.1 Qualidade de Código

Nível	Descrição
Excelente	Código limpo, modular, com docstrings, type hints, tratamento de erros
Bom	Código organizado, algumas docstrings, tratamento básico de erros
Básico	Código funcional mas desorganizado, sem documentação inline
Insuficiente	Código confuso, copy-paste, sem estrutura

9.2 Arquitetura

Nível	Descrição
Excelente	Decisões justificadas em ADRs, separação clara de responsabilidades, escalável
Bom	Arquitetura coerente, algumas decisões documentadas
Básico	Arquitetura funcional mas com acoplamentos, pouca documentação
Insuficiente	Arquitetura confusa, componentes mal integrados

9.3 Operacionalização

Nível	Descrição
Excelente	Setup automatizado, health checks, logs estruturados, monitoramento
Bom	Setup funcional, logs básicos, recuperação de falhas
Básico	Setup manual mas documentado, logs mínimos
Insuficiente	Setup complexo não documentado, sem logs

10. Template de ADR (Architecture Decision Record)

```
# ADR-001: [Título da Decisão]

## Status
[Proposto | Aceito | Deprecated | Substituído por ADR-XXX]

## Contexto
Descreva o contexto e o problema que levou a esta decisão.

## Decisão
Descreva a decisão tomada e a solução escolhida.

## Alternativas Consideradas
1. **Alternativa A**: descrição, prós, contras
2. **Alternativa B**: descrição, prós, contras

## Consequências
### Positivas
- ...

### Negativas
- ...

## Referências
- Links para documentação, artigos, etc.
```


Exemplo de ADR

```
# ADR-002: Escolha do MongoDB para Cadastro de Equipamentos

## Status
Aceito

## Contexto
Precisamos armazenar dados de equipamentos industriais com estrutura hierárquica
(equipamento → sensores → configurações). O schema pode evoluir conforme novos
tipos de sensores são adicionados.

## Decisão
Utilizar MongoDB como banco de dados para o cadastro de equipamentos, com modelo
de documento que embute sensores dentro do documento do equipamento.

## Alternativas Consideradas
1. **PostgreSQL com JSONB**:
  - Prós: já temos expertise, ACID completo
  - Contras: consultas em JSON menos naturais, schema menos flexível

2. **MongoDB com referências**:
  - Prós: normalizado, fácil atualizar sensores isoladamente
  - Contras: requer múltiplas queries, equipamento e sensores sempre lidos juntos

## Consequências
### Positivas
- Schema flexível para novos tipos de sensores
- Leitura de equipamento + sensores em uma query
- Queries naturais em dados hierárquicos

### Negativas
- Duplicação se mesmo sensor estiver em múltiplos equipamentos (não é nosso caso)
- Documento pode crescer se houver muitos sensores (limitamos a 50)
```

11. Cronograma

Semana	Conteúdo da Disciplina	Atividade do Projeto
1-2	NoSQL: conceitos, CAP, modelagem	Formar equipe, definir escopo
3-4	NoSQL: padrões de modelagem	Modelagem MongoDB, setup Docker
5	Data Lakes e Lakehouses	ENTREGA E1
6-7	Object Storage, Medallion Architecture	Implementar camadas Bronze/Silver
8-9	Table Formats, Query Engines	Jobs Spark, DAGs Airflow
10	ETL vs ELT, Incremental Load	ENTREGA E2
11-12	Lambda/Kappa, Janelas, Watermarks	Pipeline streaming
13-14	Orquestração, integração	Testes, documentação, ajustes
15	Apresentações	ENTREGA E3 + DEMO

12. Recursos e Referências

12.1 Repositórios de Exemplo

- [Data Engineering Zoomcamp](https://github.com/DataTalksClub/data-engineering-zoomcamp) ➔ <https://github.com/DataTalksClub/data-engineering-zoomcamp>
- [Awesome Data Engineering](https://github.com/igorbarinov/awesome-data-engineering) ➔ <https://github.com/igorbarinov/awesome-data-engineering>

- [Spark Examples](https://github.com/apache/spark/tree/master/examples) ➞ <https://github.com/apache/spark/tree/master/examples>

12.2 Documentação Oficial

- [Apache Kafka Documentation](https://kafka.apache.org/documentation/) ➞ <https://kafka.apache.org/documentation/>
- [Apache Airflow Documentation](https://airflow.apache.org/docs/) ➞ <https://airflow.apache.org/docs/>
- [Apache Spark Documentation](https://spark.apache.org/docs/latest/) ➞ <https://spark.apache.org/docs/latest/>
- [MinIO Documentation](https://min.io/docs/minio/linux/index.html) ➞ <https://min.io/docs/minio/linux/index.html>

12.3 Docker Compose de Referência

Um `docker-compose.yml` inicial será disponibilizado no repositório da disciplina como ponto de partida.

13. Política de Integridade Acadêmica

- **Código:** deve ser desenvolvido pela equipe. Uso de bibliotecas e snippets é permitido com devida atribuição.
- **Documentação:** textos devem ser originais. Citações devem ser referenciadas.
- **IA Generativa:** uso permitido como ferramenta de apoio, mas o entendimento do código é responsabilidade da equipe e será verificado na apresentação.
- **Plágio:** cópia de projetos de outras equipes ou da internet resultará em nota zero.

Bom trabalho! 🚀